Laporan Tugas Besar 1

IF3170 - Inteligensi Artifisial

Dipersiapkan oleh

K01 - WeDaBest



Muhammad Nur Majiid	13524028
Stefani Angeline Oroh	13524064
Angelina Andra Alanna	13524079

DAFTAR ISI

BAB I DESKRIPSI PERSOALAN **4**
1.1 Latar Belakang 4
1.2 Komponen Persoalan 4
1.3 Tujuan 5
1.4 Batasan (*Constraint*) 5
1.5 Ketentuan Pencarian 5

BAB II LANDASAN TEORI **7**
2.1 Local Search 7
2.2 State, Neighbor, dan Objective Function 7
2.3 Hill-Climbing 7
2.4 Simulated Annealing 8
2.5 Genetic Algorithm 8
2.6 Penanganan Constraint dengan Fungsi Penalti 9

BAB III METODOLOGI **10**
3.1 Tahapan Pengerjaan 10
3.2 Representasi Persoalan 10
3.2.1 Model Data 10
3.2.2 Orientasi dan Rotasi 11
3.2.3 Pengecekan Constraint 11
3.3 Fungsi Objektif 12
3.3.1 Fungsi Objektif Utama 12
3.3.2 Objective Berpenalti (SA dan GA) 13
3.3.3 Fungsi Objektif Bonus: Prioritas ETA 13
3.4 Initial State dan Neighbor 14
3.4.1 Initial State 14
3.4.2 Neighbor Acak (SA dan Mutasi GA) 14
3.4.3 Seluruh Neighbor (Semua Varian HC) 14
3.5 Algoritma 14
3.5.1 Hill-Climbing 14
3.5.2 Simulated Annealing 15
3.5.3 Genetic Algorithm 16
3.5.4 Bonus Multi-Truk 17
3.6 Heuristik yang Digunakan 18

BAB IV PERANCANGAN & IMPLEMENTASI **20**
4.1 Struktur Program 20
4.2 Modul Inti (src/models/) 20
4.2.1 model.py 21
4.2.2 geometry.py 22
4.2.3 objective.py 22
4.2.4 operators.py 23
4.2.5 io.py 23
4.3 Implementasi Hill-Climbing (HC.py) 23
4.4 Implementasi Simulated Annealing (SA.py) 24
4.5 Implementasi Genetic Algorithm (GA.py) 27

BAB V PENGUJIAN **30**
5.1 Skenario Pengujian 30
5.2 Dataset Uji 30
5.3 Data yang Dicatat 31
5.4 Hasil pada Test Case 32
5.4.1 Hill-Climbing 32
5.4.2 Simulated Annealing 34
5.4.3 Genetic Algorithm 35
5.5 Analisis Hasil 36
5.5.1 Kedekatan dengan Optimum Global 36
5.5.2 Perbandingan Hasil Antaralgoritma 37
5.5.3 Perbandingan Durasi 37
5.5.4 Konsistensi Hasil 37
5.5.5 Pengaruh Populasi dan Iterasi pada GA 37
5.5.6 Perilaku Simulated Annealing 37

BAB VI KESIMPULAN & SARAN **39**
6.1 Kesimpulan 39
6.2 Keterbatasan 39
6.3 Saran 39

DAFTAR PUSTAKA **40**
Lampiran A. Pembagian Tugas 40
Lampiran B. Form Penggunaan Generative AI 40
Lampiran C. Tautan Repository 42

BAB I
DESKRIPSI PERSOALAN

1.1. Latar Belakang

Mejiro Express adalah usaha logistik darat skala kecil milik keluarga Mejiro yang sedang berada pada tahap uji coba. Pada tahap ini hanya tersedia satu (atau beberapa) truk dengan peti kemas berukuran terbatas, sedangkan jumlah paket yang menunggu dikirim biasanya lebih banyak daripada yang bisa diangkut. Penyusunan muatan secara manual memakan waktu dan sering kali tidak optimal, sehingga dibutuhkan program yang dapat menentukan sendiri paket mana yang dimuat, di posisi mana, dan dengan orientasi apa.

Secara formal, persoalan ini merupakan gabungan *three-dimensional bin packing problem* dan *knapsack problem*. Selain mencari susunan geometris yang sah, program juga harus memilih subset paket karena tidak semua paket muat sekaligus. Ruang pencariannya sangat besar, sehingga pendekatan yang dipakai adalah *local search*.

1.2. Komponen Persoalan

Persoalan melibatkan dua entitas, yaitu *Package* (paket) dan *Truck* (truk). Atribut kedua entitas menjadi dasar representasi state, pengecekan constraint, dan perhitungan nilai solusi. Ringkasannya ditunjukkan pada Tabel 1.1.

Tabel 1.1 Entitas dan atribut persoalan

Entitas	**Atribut**	**Contoh**	**Keterangan**
Package	id	“P4”	Penanda unik paket
	Dimensions	{w:6, l:3, h:2}	Lebar, panjang, dan tinggi paket
	Orientation	{w:y, l:x, h:z}	Pemetaan sisi paket ke sumbu; ada enam kemungkinan
	Value	70	Nilai paket
	Weight	40	Berat paket
	IsFragile	True	Paket mudah pecah
	ETA	2	Sisa hari sampai paket harus tiba
Truck	Dimensions	{w:6, l:6, h:4}	Ukuran ruang peti kemas
	MaxCapacity	100	Batas total berat muatan

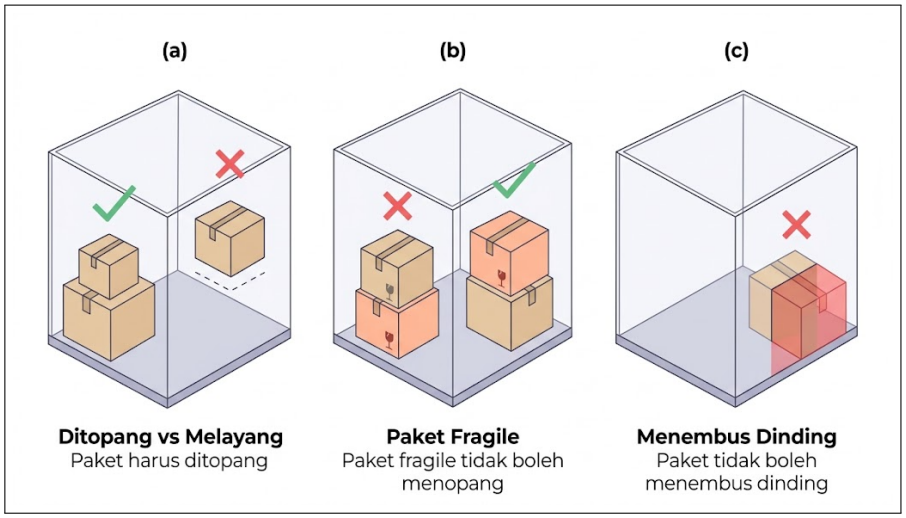
1.3. Tujuan

Tujuan persoalan adalah memaksimalkan jumlah *value* paket yang berada di dalam peti kemas. Paket yang ditinggalkan di luar truk tidak menyumbang nilai apa pun pada *objective function*.

1.4. Batasan (*Constraint*)

Suatu susunan dinyatakan valid apabila seluruh syarat berikut terpenuhi:

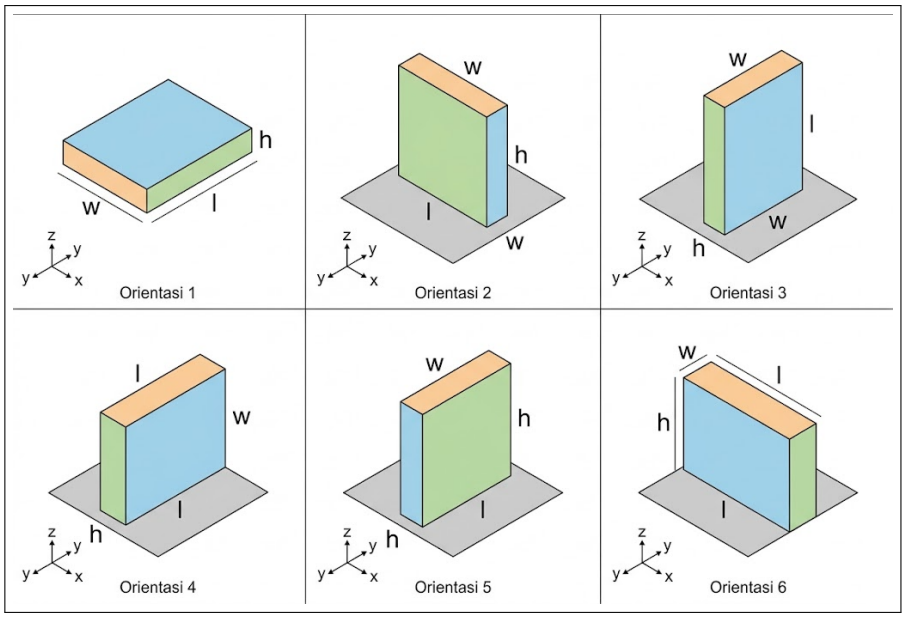
1. Paket tidak menembus dinding: setiap paket harus sepenuhnya di dalam atau sepenuhnya di luar peti kemas.
2. Paket tidak saling beririsan (*overlap*) di dalam ruang yang sama.
3. Paket tidak melayang: minimal satu titik alas paket ditopang oleh lantai atau atap paket lain.
4. Paket *fragile* tidak boleh dijadikan penopang paket lain.
5. Total berat paket dalam satu truk tidak melebihi *MaxCapacity*.



Gambar 1.1 Ilustrasi pelanggaran constraint pada penempatan paket
(Sumber: dihasilkan dengan Google Gemini)

1.5. Ketentuan Pencarian

Seluruh koordinat bernilai bilangan bulat dan *initial state* dibangkitkan secara acak. *Neighbor state* dibentuk dengan satu dari tiga jenis *move*: menukar dua paket, memindahkan satu paket ke koordinat lain, atau merotasi satu paket sebesar 90° terhadap sumbu x, y, atau z. Algoritma yang diminta adalah Hill-Climbing (Steepest Ascent, Sideways Move, Random Restart, dan Stochastic), Simulated Annealing, dan Genetic Algorithm.



Gambar 1.2 Enam kemungkinan orientasi sebuah paket
(Sumber: dihasilkan dengan Google Gemini)

BAB II
LANDASAN TEORI

2.1. Local Search

Local search bekerja dengan memegang satu state (atau sekumpulan kecil state) dan memperbaikinya secara bertahap dengan berpindah ke state tetangga. Berbeda dengan pencarian klasik seperti BFS atau A*, jalur menuju solusi tidak disimpan karena yang dicari hanyalah konfigurasi akhirnya [1].

Kebutuhan memori *local search* sangat kecil dan algoritma ini mampu memberikan solusi yang cukup baik dalam waktu wajar, sehingga cocok untuk ruang state berukuran besar seperti penempatan paket tiga dimensi. Konsekuensinya, solusi optimal global tidak dijamin ditemukan.

2.2. State, Neighbor, dan Objective Function

Local search tersusun atas tiga komponen [2]:

- **State**: satu konfigurasi lengkap dari persoalan.
- **Neighbor**: state lain yang dapat dicapai dengan satu perubahan kecil.
- **Objective function**: fungsi yang memberi skor pada sebuah state; pada persoalan maksimasi, skor yang lebih tinggi lebih baik.

Seluruh state beserta skornya membentuk *state-space landscape*. Beberapa bentuk landscape yang menyulitkan pencarian adalah *local maximum* (puncak lokal yang bukan puncak global), *plateau* atau *shoulder* (daerah datar), dan *ridge* (deretan puncak lokal yang sulit dilalui dengan satu langkah).

2.3. Hill-Climbing

Hill-Climbing selalu bergerak ke neighbor yang lebih baik dan berhenti ketika tidak ada lagi neighbor yang lebih baik [3]. Sifatnya yang *greedy* membuat algoritma ini cepat, tetapi mudah tersangkut di local maximum, plateau, atau ridge. Tabel 2.1 merangkum varian yang dipakai.

Tabel 2.1 Varian Hill-Climbing

Varian	**Prinsip**	**Berhenti ketika**
Steepest Ascent	Memeriksa seluruh neighbor dan pindah ke yang terbaik	Tidak ada neighbor yang lebih baik
Sideways Move	Seperti Steepest Ascent, tetapi boleh pindah ke neighbor bernilai sama	Langkah datar berturut-turut mencapai batas
Stochastic	Memilih secara acak salah satu neighbor yang lebih baik	Tidak ada neighbor yang lebih baik atau iterasi habis
Random Restart	Mengulang Hill-Climbing dari initial state acak dan menyimpan hasil terbaik	Jumlah restart mencapai batas

2.4. Simulated Annealing

Simulated Annealing meniru proses pendinginan logam secara perlahan agar strukturnya stabil [4]. Pada setiap iterasi dipilih satu neighbor acak. Neighbor yang lebih baik selalu diterima, sedangkan neighbor yang lebih buruk diterima dengan peluang

$$P = e^{\Delta E/T},$$

dengan $\Delta E \leq 0$ adalah selisih objective neighbor terhadap state saat ini dan T adalah suhu. Saat T tinggi, peluang menerima langkah buruk besar sehingga algoritma leluasa menjelajah. Seiring suhu turun menurut *cooling schedule*, peluang tersebut mengecil dan perilaku algoritma makin mirip Hill-Climbing.

2.5. Genetic Algorithm

Genetic Algorithm adalah algoritma berbasis populasi yang meniru seleksi alam [5]. Setiap individu mewakili satu state dan kualitasnya diukur dengan *fitness function*. Alurnya adalah:

1. membangkitkan populasi awal secara acak;
2. memilih induk dengan peluang yang sebanding dengan fitness;
3. melakukan *crossover* untuk menggabungkan informasi dua induk;
4. melakukan *mutasi* kecil untuk menjaga keragaman;
5. membentuk generasi baru dan mengulang langkah 2–4 sampai jumlah generasi tercapai.

Ukuran populasi dan jumlah generasi merupakan dua parameter utama: keduanya memperluas eksplorasi, tetapi juga menambah waktu komputasi.

2.6. Penanganan Constraint dengan Fungsi Penalti

Fungsi penalti memungkinkan algoritma mengunjungi state yang tidak valid dengan cara mengurangi objective sebesar bobot penalti dikali besar pelanggaran. Apabila bobot penalti lebih besar daripada nilai maksimum yang mungkin dicapai, setiap state valid pasti lebih baik daripada setiap state tidak valid. Landscape tetap tersambung, sehingga pencarian tidak terputus hanya karena satu langkah menghasilkan state tidak valid.

BAB III
METODOLOGI

3.1. Tahapan Pengerjaan

Pengerjaan dibagi menjadi enam tahap:

1. Membaca spesifikasi dan merumuskan komponen, constraint, serta tujuan persoalan.
2. Merancang representasi state, objective, dan operator neighbor yang dipakai bersama oleh ketiga algoritma.
3. Mengimplementasikan modul inti di `src/models/`.
4. Mengimplementasikan algoritma di `src/algoritma/`, program utama `main.py`, dan visualisasi di `src/visualisasi/`.
5. Menjalankan eksperimen pada dataset wajib `test_case` dengan seed 1, 2, dan 3.
6. Menganalisis dan membandingkan hasil.

3.2. Representasi Persoalan

3.2.1. Model Data

Data yang tidak pernah berubah (atribut truk dan paket) dipisahkan dari data yang diubah oleh algoritma (posisi, orientasi, dan truk tujuan setiap paket). Dengan pemisahan ini, penyalinan state saat membangkitkan neighbor cukup menyalin bagian yang berubah saja.

Tabel 3.1 Kelas pada model data

Kelas	**Isi**	**Diubah algoritma?**
`Truck`	Dimensi (W, L, H) dan `max_capacity`	Tidak
`Package`	`id`, dimensi asli (w, l, h), `value`, `weight`, `is_fragile`, `eta`	Tidak
`Placement`	Referensi `Package`, `position` (x, y, z) atau `None` jika di luar truk, `orientation_idx` (0–5), dan `truck_idx`	Ya
`State`	Daftar `Truck` dan satu `Placement` untuk setiap paket	Ya

Paket di luar truk ditandai dengan `position = None`. Dengan representasi ini, tidak ada keadaan "setengah di dalam": paket yang koordinatnya menembus dinding dihitung sebagai pelanggaran, bukan sebagai paket di luar truk. Koordinat (x, y, z) menyatakan sudut kiri-bawah-belakang paket setelah orientasi diterapkan, dengan titik $(0, 0, 0)$ di pojok truk. Sumbu x sejajar W, sumbu y sejajar L, dan sumbu z sejajar H. Pada kasus multi-truk, setiap truk memiliki sistem koordinat sendiri.

3.2.2. Orientasi dan Rotasi

Orientasi direpresentasikan sebagai permutasi dimensi asli paket ke sumbu (x, y, z), sehingga terdapat enam orientasi (Tabel 3.2).

Tabel 3.2 Enam orientasi paket

Indeks	**Permutasi**	**Dimensi efektif** (x, y, z)	**Label**
0	(0, 1, 2)	(w, l, h)	{w:x, l:y, h:z}
1	(0, 2, 1)	(w, h, l)	{w:x, l:z, h:y}
2	(1, 0, 2)	(l, w, h)	{w:y, l:x, h:z}
3	(1, 2, 0)	(l, h, w)	{w:z, l:x, h:y}
4	(2, 0, 1)	(h, w, l)	{w:y, l:z, h:x}
5	(2, 1, 0)	(h, l, w)	{w:z, l:y, h:x}

Rotasi 90° terhadap sebuah sumbu sama dengan menukar dua komponen permutasi yang tegak lurus terhadap sumbu tersebut: rotasi terhadap x menukar $y \leftrightarrow z$, terhadap y menukar $x \leftrightarrow z$, dan terhadap z menukar $x \leftrightarrow y$. Keenam orientasi dapat dicapai dari orientasi mana pun melalui rangkaian rotasi tersebut.

3.2.3. Pengecekan Constraint

Setiap constraint diperiksa per truk, sehingga paket di truk berbeda tidak pernah dianggap beririsan atau saling menopang.

Tabel 3.3 Definisi pengecekan constraint

Constraint	**Definisi dalam kode**	**Satuan pelanggaran**
Dinding (`bounds`)	$x, y, z \geq 0$ dan $x + w \leq W$, $y + l \leq L$, $z + h \leq H$	Per paket
Overlap	Irisan *axis-aligned bounding box* positif pada ketiga sumbu; bersentuhan sisi tidak dihitung	Per pasangan
Kapasitas (`weight`)	$\sum$ berat paket di truk $k \leq$ kapasitas truk k	Per truk
Fragile	Ada paket yang alasnya tepat di atap paket fragile dan irisan bidang xy keduanya positif	Per pasangan
Melayang (`support`)	Paket dengan $z > 0$ wajib punya minimal satu titik alas di atas atap paket non-fragile	Per paket

Paket fragile diperlakukan konsisten di dua tempat: tidak boleh ada paket di atasnya (constraint fragile) dan tidak dihitung sebagai penopang saat pengecekan melayang.

3.3. Fungsi Objektif

3.3.1. Fungsi Objektif Utama

$$value(s) = \sum_{i \in \text{dalam truk}} value_i$$

Nilai ini tidak memperhitungkan validitas. Validitas ditangani berbeda oleh setiap algoritma (Tabel 3.4).

Tabel 3.4 Penanganan state tidak valid

Algoritma	**Penanganan**
Hill-Climbing	Hanya neighbor valid yang dipertimbangkan, sehingga objective cukup $value(s)$
Simulated Annealing dan Genetic Algorithm	State tidak valid boleh dikunjungi dengan objective berpenalti (subbab 3.3.2)

3.3.2. Objective Berpenalti (SA dan GA)

$$P(s) = V_{bounds} + V_{overlap} + V_{support} + V_{fragile} + V_{weight}$$

$$V_{weight} = \sum_{k \in \text{truk overload}} \left(1 + \frac{berat_k - kapasitas_k}{kapasitas_k}\right)$$

$$objective(s) = value(s) - \lambda \cdot P(s), \qquad \lambda = \sum_{\text{semua paket}} value_i + 1$$

- V_{bounds} dan $V_{support}$ dihitung per paket, sedangkan $V_{overlap}$ dan $V_{fragile}$ per pasangan paket.
- λ lebih besar daripada value maksimum, sehingga state valid apa pun selalu lebih baik daripada state tidak valid.
- Di antara state tidak valid, pelanggaran yang lebih sedikit memberi objective lebih tinggi sehingga pencarian punya arah kembali ke daerah valid.
- V_{weight} berbentuk pecahan agar pengurangan kelebihan berat tetap dihargai sebagai perbaikan.

Karena SA dan GA boleh mengunjungi state tidak valid, kedua algoritma menyimpan dua state hasil:

- `final_state`: pada SA, state tempat pencarian berhenti; pada GA, individu dengan fitness berpenalti tertinggi sepanjang run.
- `best_valid_state` (disingkat `best_valid`): state valid dengan value tertinggi yang pernah ditemui selama pencarian.

Perbandingan kualitas solusi antaralgoritma memakai `best_valid`, sedangkan `final_state` tetap disimpan untuk menunjukkan kondisi algoritma di akhir pencarian. Keduanya bisa berbeda. Contohnya, SA seed 1 pada test_case memiliki `final_state` bernilai 200, tetapi pernah menemukan state valid bernilai 205 (Tabel 5.4).

3.3.3. Fungsi Objektif Bonus: Prioritas ETA

$$score_{eta}(s) = \sum_{i \in \text{dalam truk}} value_i \times \left(1 + \frac{1}{ETA_i}\right)$$

Paket dengan ETA kecil mendapat bobot lebih besar (2 untuk ETA = 1, 1,5 untuk ETA = 2). Paket dengan ETA kosong atau tidak positif diberi bobot 1. Saat OF ini dipilih, λ disesuaikan menjadi $\sum score_{eta}$ seluruh paket $+1$ dan hasil disimpan terpisah di `output/objective_eta/`.

3.4. Initial State dan Neighbor

3.4.1. Initial State

Initial state dibangkitkan dengan mengacak urutan paket. Untuk setiap paket dicoba hingga 100 kombinasi acak (orientasi, truk, dan posisi di dalam dinding). Penempatan pertama yang menjaga state tetap valid langsung dipakai; jika seluruh percobaan gagal, paket diletakkan di luar truk. Oleh karena itu initial state selalu valid, yang merupakan syarat Hill-Climbing.

3.4.2. Neighbor Acak (SA dan Mutasi GA)

Fungsi `make_random_neighbor` memilih satu dari tiga move dengan peluang sama.

Tabel 3.5 Operator neighbor

Move	Efek
Swap	Menukar posisi, orientasi, dan truk dua paket acak (termasuk paket dalam dengan paket luar, atau antartruk)
Relocate	15%: paket dikeluarkan dari truk; 85%: paket dipindah ke truk dan posisi acak di dalam dinding
Rotate	Merotasi satu paket 90° terhadap sumbu acak

Jika hanya ada satu paket, swap tidak mungkin dilakukan sehingga move diganti menjadi rotate.

3.4.3. Seluruh Neighbor (Semua Varian HC)

Fungsi `iter_all_feasible_neighbors` membangkitkan seluruh successor valid dari satu move: swap untuk setiap pasangan paket ($n(n-1)/2$ kandidat), relocate setiap paket ke luar truk dan ke setiap koordinat bulat yang muat, serta rotate setiap paket terhadap tiga sumbu ($3n$ kandidat). Kandidat yang sama dengan state asal atau duplikat (dicek dengan `state_signature`) dibuang. Generator ini dipakai oleh keempat varian HC, termasuk Stochastic.

Seluruh keacakan memakai satu objek `random.Random(seed)`, sehingga seed yang sama selalu menghasilkan run yang sama.

3.5. Algoritma

3.5.1. Hill-Climbing

Keempat varian hanya berpindah ke neighbor valid dengan objective $value(s)$ (atau $score_{eta}$ jika OF bonus dipilih).

Tabel 3.6 Varian Hill-Climbing yang diimplementasikan

Varian	Langkah per iterasi	Kondisi berhenti	Parameter
Steepest Ascent	Mengevaluasi seluruh neighbor valid dan pindah ke yang terbaik; seri dipecah acak (*reservoir sampling*)	`LOCAL_OPTIMUM`, `NO_FEASIBLE_NEIGHBOR`, atau `max_iter`	`max_iter` = 1000
Sideways Move	Seperti Steepest Ascent, tetapi neighbor terbaik yang bernilai sama juga diterima	`MAX_SIDEWAYS`, `LOCAL_OPTIMUM`, atau `max_iter`	`max_iter` = 1000, `max_consecutive_sideways` = 50
Stochastic	Mengumpulkan semua neighbor yang lebih baik (*uphill*) lalu memilih satu secara acak seragam	`LOCAL_OPTIMUM` atau `max_iter`	`max_iter` = 1000
Random Restart	Steepest Ascent dari initial state, lalu diulang dari initial state acak baru; hasil terbaik disimpan	`MAX_RESTARTS` atau `ALL_PACKAGES_LOADED`	`max_iter_per_restart` = 1000, `max_restart` = 10

Pada Sideways Move, penghitung langkah datar di-*reset* setiap kali terjadi langkah naik, sehingga yang dibatasi adalah langkah datar berturut-turut. Pada Random Restart, pencarian awal (dari initial state yang diberikan) dicatat terpisah sebagai `initial_search` dengan label restart 0, sedangkan `restart_count` hanya menghitung restart sesungguhnya (maksimal 10).

3.5.2. Simulated Annealing

```
current ← initial_state
untuk t = 0, 1, 2, ...:
    T ← T0 · α^t                    (pendinginan geometrik)
    jika T < T_min atau t ≥ max_iter: berhenti
    neighbor ← neighbor acak, diulang maks max_neighbor_tries kali sampai valid
    ΔE ← objective(neighbor) - objective(current)
    jika ΔE > 0: terima
    jika ΔE ≤ 0: terima jika random() < e^(ΔE/T)
    jika diterima: current ← neighbor; perbarui best_valid
    stuck: +1 setiap stuck_window iterasi berturut-turut tanpa objective terbaik
    naik
```

Penerimaan langkah buruk bersifat probabilistik sesuai materi kuliah: langkah turun diterima jika bilangan acak $r \in [0,1)$ lebih kecil dari $e^{\Delta E/T}$. Untuk $\Delta E > 0$ peluang dicatat sebagai 1, dan eksponen di atas 709 dibatasi agar tidak terjadi *overflow*.

Tabel 3.7 Parameter Simulated Annealing

Parameter	Nilai	Keterangan
`initial_temperature` (T_0)	100	Suhu awal
`cooling_rate` (α)	0,999	Faktor pendinginan geometrik
`min_temperature`	0,01	SA selalu berhenti di iterasi ke-9.206 karena suhu
`max_iter`	20.000	Tidak pernah tercapai dengan parameter di atas
`stuck_window`	500	Panjang jendela tanpa perbaikan
`max_neighbor_tries`	50	Batas percobaan mencari neighbor valid

Selain `stuck_count`, dicatat juga `stuck_rate`, yaitu jumlah kejadian stuck per 1.000 iterasi, agar frekuensi stuck dapat dibandingkan antar-run yang panjangnya berbeda.

3.5.3. Genetic Algorithm

Setiap individu adalah satu `State` lengkap. Kromosomnya adalah daftar `Placement` dengan satu gen per paket, dan setiap gen berisi posisi, orientasi, serta truk paket tersebut. Fitness memakai objective berpenalti yang sama dengan SA (subbab 3.3.2), sehingga individu tidak valid tetap boleh ada di populasi tetapi hampir tidak pernah mengalahkan individu valid. Alur satu generasi adalah sebagai berikut.

1. **Evaluasi**: fitness seluruh individu dihitung, lalu individu terbaik dan `best_valid` diperbarui.
2. **Elitisme**: `elitisme` = 2 individu dengan fitness tertinggi disalin langsung ke generasi berikutnya.
3. **Seleksi**: dua induk dipilih dengan *roulette wheel*. Fitness digeser terhadap nilai minimum populasi agar semua bobot positif, karena fitness berpenalti bisa negatif.
4. **Crossover**: dengan peluang `crossover_rate` = 0,81 dilakukan *uniform crossover*: untuk setiap paket, gen diambil dari induk pertama atau kedua dengan peluang masing-masing 0,5. Gen dicocokkan berdasarkan `id` paket, sehingga anak selalu berisi tepat satu placement untuk setiap paket. Jika crossover tidak terjadi, anak adalah salinan induk pertama.
5. **Mutasi**: dengan peluang `mutation_rate` = 0,2, anak dikenai 1–3 move acak dari `make_random_neighbor` (swap, relocate, atau rotate).

6. Langkah 3–5 diulang sampai ukuran populasi kembali penuh.

GA tidak memiliki kriteria berhenti dini. Pencarian selalu berjalan tepat `max_iter` generasi, sehingga jumlah populasi dan iterasi menjadi parameter yang diuji pada eksperimen.

Tabel 3.8 Parameter Genetic Algorithm

Parameter	Nilai	Keterangan
`population_size`	20 / 50 / 100	Divariasikan pada eksperimen B (iterasi 100)
`max_iter`	50 / 100 / 200	Jumlah generasi; divariasikan pada eksperimen A (populasi 50)
`crossover_rate`	0,81	Peluang crossover per pasangan induk
`mutation_rate`	0,2	Peluang seorang anak dimutasi
`mutation_moves`	3	Batas atas jumlah move per mutasi (nilai bawaan, tidak diatur di `experiments.json`)
`elitisme`	2	Individu terbaik yang dibawa tanpa perubahan

Ada dua hasil yang disimpan. `final_state` adalah individu dengan fitness berpenalti tertinggi sepanjang run, dan `initial_state` adalah individu terbaik generasi 0. Hasil yang dilaporkan adalah `best_valid`, yaitu individu valid dengan value tertinggi yang pernah muncul. Jika `max_iter` = 0, loop tidak berjalan dan GA langsung mengambil individu terbaik dari populasi awal.

3.5.4. Bonus Multi-Truk

Multi-truk ditangani di tingkat representasi, bukan di tingkat algoritma. Setiap `Placement` menyimpan `truck_idx`, dan setiap truk memiliki sistem koordinat sendiri. Dukungan multi-truk tersebar pada komponen berikut:

- **Dataset**: `load_dataset` menerima format satu truk (`"truck"`) maupun daftar truk (`"trucks"`); format satu truk diperlakukan sebagai daftar berisi satu truk.
- **Initial state**: setiap percobaan penempatan memilih truk secara acak.
- **Neighbor acak**: relocate memilih truk tujuan secara acak, sedangkan swap ikut menukar `truck_idx` sehingga dua paket dapat bertukar truk.

- **Seluruh neighbor (HC)**: relocate mencoba setiap koordinat di *setiap* truk yang cukup besar untuk paket tersebut.
- **Constraint**: overlap, fragile, dan support hanya diperiksa antarpaket pada truk yang sama, sedangkan kapasitas dihitung per truk terhadap `max_capacity` masing-masing.

Karena itu HC, SA, dan GA dapat menyelesaikan kasus multi-truk tanpa perubahan kode algoritma. Sebagai uji coba, GA dijalankan pada `multi_truck_case` (dua truk, kapasitas 100 dan 80, dengan delapan paket yang sama seperti `test_case`). Dengan populasi 10 dan 10 iterasi, GA memperoleh value 305, yang berarti seluruh paket berhasil dimuat ke dua truk.

3.6. Heuristik yang Digunakan

- **Initial state selalu valid.** Paket hanya ditambahkan jika state tetap valid, sehingga HC dapat langsung berjalan dan SA/GA mulai dari daerah valid.
- **Mengeluarkan paket saat relocate (15%).** Pada neighbor acak, relocate memiliki peluang 15% untuk mengeluarkan paket dari truk. Langkah ini membuka ruang bagi paket lain ketika susunan sudah terkunci.
- **Neighbor duplikat dibuang.** Dalam satu neighborhood, kandidat yang sama dengan state asal atau dengan kandidat sebelumnya (dicek dengan `state_signature`) tidak dievaluasi ulang. Pengecekan ini hanya berlaku dalam satu iterasi, sehingga Sideways Move tetap dapat kembali ke state yang pernah dikunjungi; yang membatasinya adalah `max_consecutive_sideways`.
- **Seri dipecah secara acak.** Jika beberapa neighbor terbaik bernilai sama, salah satunya dipilih acak dengan *reservoir sampling*, sehingga HC tidak selalu bergerak ke arah yang sama.
- **Bobot penalti λ = total value + 1.** State valid apa pun selalu lebih baik daripada state tidak valid, tetapi state tidak valid masih dapat dibandingkan berdasarkan banyaknya pelanggaran.
- **Penalti berat berbentuk pecahan.** Mengurangi kelebihan berat tetap dihargai sebagai perbaikan walaupun truk belum berada di bawah kapasitas.
- **Neighbor valid untuk SA.** SA mencoba hingga `max_neighbor_tries` = 50 kali untuk mendapatkan neighbor valid. Pada test_case, state current SA tetap valid di seluruh iterasi.

- **Penyimpanan** `best_valid`**.** Solusi bagus yang sempat ditinggalkan SA pada suhu tinggi, atau yang hilang di GA akibat crossover, tetap tersimpan sebagai hasil.

BAB IV
PERANCANGAN & IMPLEMENTASI

4.1. Struktur Program

Program ditulis dalam Python dengan satu dependensi eksternal, yaitu `matplotlib` untuk visualisasi. Kode terbagi menjadi modul inti yang dipakai bersama, algoritma, serta program utama dan visualisasi.

```
src/
  main.py               # program utama (menu CLI)
  models/
    model.py            # Package, Truck, Placement, State, orientasi
    geometry.py         # pengecekan lima constraint
    objective.py        # OF value dan OF bonus eta
    operators.py        # initial state dan operator neighbor
    io.py               # baca dataset, simpan/muat state, history, config
  algoritma/
    HC.py               # empat varian Hill-Climbing
    SA.py               # Simulated Annealing
    GA.py               # Genetic Algorithm
  visualisasi/
    plots.py            # plot objective, e^(dE/T), dan state 3D
    export_replay.py    # rekam state per langkah untuk viewer
    viewer3d.html       # viewer 3D interaktif (Three.js)
  data/                 # dataset dan experiments.json
tests/
  test_hc.py  test_sa.py  test_ga.py
```

Gambar 4.1 Struktur direktori program

HC dan SA menerima `run(initial_state, rng, **params)`, sedangkan GA menerima `run(trucks, packages, rng, population_size, max_iter, **params)` karena GA membangkitkan populasinya sendiri. Ketiganya mengembalikan `dict` dengan kunci yang seragam (`initial_state`, `final_state`, `best_valid_state`, `final_objective`, `iterations`, `duration_s`, `history`, `extra`), sehingga penyimpanan hasil dan visualisasi dapat dipakai bersama. Parameter eksperimen dibaca dari `data/experiments.json`.

4.2. Modul Inti (src/models/)

Modul inti berisi model data, pengecekan constraint, fungsi objektif, operator neighbor, dan I/O yang dipakai bersama oleh ketiga algoritma.

4.2.1. model.py

Tabel 4.1 Isi model.py

Nama	Jenis	Deskripsi
`ORIENTATIONS`	Konstanta	Enam permutasi dimensi beserta labelnya
`get_effective_dimensions`	Fungsi	Dimensi paket pada sumbu (x, y, z) untuk suatu orientasi
`rotate_orientation_idx`	Fungsi	Indeks orientasi baru setelah rotasi 90°
`Package`, `Truck`	Kelas	Data tetap paket dan truk
`Placement`	Kelas	Posisi, orientasi, truk; menyediakan `effective_dims` dan `aabb()`
`State`	Kelas	Daftar truk dan placement; `total_value()`, `truck_weight()`, `copy()`, konversi JSON

```
def rotate_orientation_idx(current_idx: int, axis: str) -> int:
    perm = list(ORIENTATIONS[current_idx])
    if axis == "x":
        perm[1], perm[2] = perm[2], perm[1]
    elif axis == "y":
        perm[0], perm[2] = perm[2], perm[0]
    else:  # z
        perm[0], perm[1] = perm[1], perm[0]
    return _PERM_TO_IDX[tuple(perm)]
```

4.2.2. geometry.py

Tabel 4.2 Fungsi pada geometry.py

Fungsi	Deskripsi
`is_inside_bounds`, `check_bounds`	Constraint dinding
`placements_overlap`, `check_no_overlap`	Constraint overlap (uji AABB per truk)
`check_weight`, `overweight_score`	Constraint kapasitas dan besar kelebihan berat
`check_fragile_support`	Constraint paket di atas paket fragile
`check_support`	Constraint melayang
`validate`, `is_feasible`	Menjalankan kelima pengecekan dan mengembalikan pelanggaran per kategori

Dua balok tidak beririsan jika keduanya terpisah pada minimal satu sumbu. Penggunaan `<=` membuat paket yang hanya bersentuhan sisi tidak dianggap overlap:

```
def placements_overlap(a: Placement, b: Placement) -> bool:
    ax1, ay1, az1, ax2, ay2, az2 = a.aabb()
    bx1, by1, bz1, bx2, by2, bz2 = b.aabb()
    if ax2 <= bx1 or bx2 <= ax1: return False
    if ay2 <= by1 or by2 <= ay1: return False
    if az2 <= bz1 or bz2 <= az1: return False
    return True
```

4.2.3. objective.py

Definisi objective dipisahkan dari algoritma, sehingga OF bonus dapat dipakai oleh ketiga algoritma tanpa mengubah kodenya.

```
def eta_weight(package: Package) -> float:
    if package.eta is None or package.eta <= 0:
        return 1.0
    return 1.0 + 1.0 / package.eta
```

4.2.4. operators.py

Tabel 4.3 Fungsi pada operators.py

Fungsi	Deskripsi
`make_initial_state`	Initial state acak yang selalu valid
`swap_packages`, `relocate_package`, `rotate_package`	Tiga move dasar; mengembalikan salinan state baru
`relocate_random`, `rotate_random`	Versi acak relocate dan rotate
`make_random_neighbor`	Satu move acak (SA dan mutasi GA)
`iter_all_feasible_neighbors`	Generator seluruh neighbor valid unik (semua varian HC)
`state_signature`	Representasi state yang dapat di-*hash*

4.2.5. io.py

`load_dataset` membaca dataset satu truk maupun multi-truk, `save_state`/`load_state` menyimpan dan memuat state, `save_history_csv` menyimpan riwayat per iterasi, dan `load_config` membaca `experiments.json`.

4.3. Implementasi Hill-Climbing (HC.py)

Tabel 4.4 Fungsi pada HC.py

Fungsi	Deskripsi
`run_steepest`	Steepest Ascent
`run_sideways`	Sideways Move
`run_stochastic`	Stochastic: pilih acak di antara neighbor yang lebih baik
`run_random_restart`	Random Restart (memanggil `run_steepest` berulang kali)
`_best_feasible_neighbor`	Neighbor valid terbaik dari seluruh neighborhood

Inti Stochastic Hill-Climbing:

```
uphill = []
for neighbor in iter_all_feasible_neighbors(current):
    value = state_score(neighbor, objective)
    if value > current_value:
        uphill.append((neighbor, value))
if not uphill:
    termination_reason = "LOCAL_OPTIMUM"
    break
current, current_value = uphill[rng.randrange(len(uphill))]
```

4.4. Implementasi Simulated Annealing (SA.py)

Tabel 4.5 Fungsi pada SA.py

Fungsi	**Deskripsi**
`penalty`	Jumlah pelanggaran kelima constraint, $P(s)$
`objective_function`	Objective berpenalti $value(s) - \lambda \cdot P(s)$
`schedule`	Suhu pada iterasi ke-t, yaitu $T_0 \cdot \alpha^t$
`acceptance_probability`	Menghitung $e^{\Delta E/T}$ dengan pengaman *overflow*
`make_valid_neighbor`	Neighbor acak yang diulang sampai valid, maksimal `max_neighbor_tries` kali
`run`	Loop utama Simulated Annealing
`params_from_config`	Membaca parameter dari bagian `simulated_annealing` pada `experiments.json`
`save_result`	Menyimpan state awal/akhir, `history.csv`, dan `summary.json`

Penalti dan objective berpenalti diimplementasikan langsung dari rumus pada subbab 3.3.2. Nilai λ diambil dari `max_score`, sehingga otomatis menyesuaikan saat OF bonus dipakai:

```
def penalty(state: State) -> float:
    violations = validate(state)
    return (len(violations["bounds"]) + len(violations["overlap"])
            + len(violations["support"]) + len(violations["fragile"])
            + overweight_score(state))

def objective_function(state: State, objective: str = "value") -> float:
    lambda_penalty = max_score(state, objective) + 1
    return state_score(state, objective) - lambda_penalty * penalty(state)
```

Suhu turun secara geometrik, dan peluang penerimaan dibatasi agar `math.exp` tidak *overflow* ketika $\Delta E/T$ sangat besar (e^{709} mendekati batas `float`):

```
def schedule(iteration, initial_temperature, cooling_rate):
    return initial_temperature * (cooling_rate ** iteration)

def acceptance_probability(delta_e, temperature):
    ratio = delta_e / temperature
    if ratio > 709.0:
        return float("inf")
    return math.exp(ratio)
```

Neighbor dicari dengan `make_random_neighbor` sampai valid. Jika dalam `max_neighbor_tries` percobaan tidak ada yang valid, kandidat terakhir tetap dipakai dan penalti pada objective yang akan membuatnya hampir pasti ditolak:

```
def make_valid_neighbor(state, rng, max_tries):
    for tries in range(1, max_tries + 1):
        neighbor = make_random_neighbor(state, rng)
        if is_feasible(neighbor)[0]:
            break
    return neighbor, tries
```

Inti loop `run` adalah sebagai berikut. Langkah naik selalu diterima, sedangkan langkah datar atau turun diterima dengan membandingkan bilangan acak terhadap $e^{\Delta E/T}$:

```
while iteration < max_iter:
    temperature = schedule(iteration, initial_temperature, cooling_rate)
    if temperature < min_temperature:
        break
    neighbor, tries = make_valid_neighbor(current, rng, max_neighbor_tries)
    neighbor_obj = objective_function(neighbor, objective)
    delta_e = neighbor_obj - current_obj

    if delta_e > 0:
        accept_prob = 1.0
        accepted = True
    else:
        accept_prob = acceptance_probability(delta_e, temperature)
        accepted = rng.random() < accept_prob

    if accepted:
        current, current_obj = neighbor, neighbor_obj
        current_feasible = is_feasible(current)[0]
        if current_feasible and (best_valid_state is None or
                state_score(current, objective) > state_score(best_valid_state,
    objective)):
            best_valid_state = current.copy()
    ...
```

```
    iteration += 1
```

Frekuensi stuck dihitung di dalam loop yang sama. Penghitung `no_improve_local` bertambah setiap iterasi yang tidak memperbaiki objective terbaik. Saat mencapai `stuck_window`, `stuck_count` bertambah satu dan penghitung di-*reset*:

```
if current_obj > best_obj:
    best_obj = current_obj
    no_improve_local = 0
else:
    no_improve_local += 1
    if no_improve_local >= stuck_window:
        stuck_count_global += 1
        no_improve_local = 0
```

Setiap iterasi mencatat suhu, objective, value, status valid, ΔE, peluang penerimaan, status diterima, dan jumlah percobaan neighbor ke `history`. Data ini dipakai untuk membuat `objective.png` dan `accept_prob.png`. Setelah loop selesai, `stuck_rate` dihitung sebagai `stuck_count` per 1.000 iterasi.

4.5. Implementasi Genetic Algorithm (GA.py)

Tabel 4.6 Fungsi pada GA.py

Fungsi	Deskripsi
`penalty`, `objective_function`	Fitness berpenalti, sama dengan SA
`make_population`	Populasi awal dari `make_initial_state` (semua valid)
`roulette`	Seleksi *roulette wheel* dengan fitness yang digeser ke positif
`crossover`	*Uniform crossover* per paket
`mutate`	1–3 move acak dengan peluang `mutation_rate`
`best_by_fitness`	Individu dengan fitness tertinggi pada suatu populasi
`best_valid`	Individu valid dengan value tertinggi pada suatu populasi
`run`	Loop generasi GA
`experiment_matrix`	Daftar konfigurasi eksperimen A dan B dari `experiments.json`
`save_result`	Menyimpan state, `history.csv`, dan `summary.json`

Fitness pada objective berpenalti bisa bernilai negatif, sehingga sebelum seleksi semua nilai digeser terhadap fitness terkecil. Dengan begitu setiap individu tetap memiliki bobot positif:

```
def roulette(population, fitnesses, rng):
    floor = min(fitnesses)
    weights = [f - floor + 1e-6 for f in fitnesses]
    threshold = rng.random() * sum(weights)
    cumulative = 0.0
    for individual, weight in zip(population, weights):
        cumulative += weight
        if cumulative >= threshold:
            return individual
    return population[-1]
```

Kromosom adalah daftar `Placement`, satu gen per paket. Crossover memilih gen dari salah satu induk dengan peluang 0,5 per paket. Karena gen dicocokkan berdasarkan `id` paket, anak selalu berisi tepat satu placement untuk setiap paket. Mutasi memakai operator neighbor yang sama dengan SA:

```
def crossover(parent_a, parent_b, rng, crossover_rate):
    if rng.random() >= crossover_rate:
        return parent_a.copy()
    donor = {p.package.id: p for p in parent_b.placements}
    child_placements = []
    for placement in parent_a.placements:
        source = placement if rng.random() < 0.5 else donor[placement.package.id]
        child_placements.append(source.copy())
    return State(trucks=list(parent_a.trucks), placements=child_placements)

def mutate(individual, rng, mutation_rate, max_moves=3):
    child = individual.copy()
    if rng.random() >= mutation_rate:
        return child
    for _ in range(rng.randint(1, max_moves)):
        child = make_random_neighbor(child, rng)
    return child
```

Pada setiap generasi, `run` menghitung fitness, mencatat statistik populasi, memperbarui individu terbaik dan `best_valid`, lalu membentuk generasi baru. Elitisme dilakukan dengan menyalin `elitisme` individu terbaik terlebih dahulu, kemudian sisa populasi diisi anak hasil seleksi, crossover, dan mutasi:

```
for iteration in range(max_iter):
    fitnesses = [objective_function(ind, objective) for ind in population]
    values = [ind.total_value() if is_feasible(ind)[0] else 0
              for ind in population]
    # perbarui best_individual dan best_valid_state, catat history
    ...
    if iteration == max_iter - 1:
        break
    elite_indices = sorted(range(len(population)),
                           key=lambda i: fitnesses[i], reverse=True)[:elitisme]
    next_population = [population[i].copy() for i in elite_indices]
    while len(next_population) < population_size:
        parent_a = roulette(population, fitnesses, rng)
        parent_b = roulette(population, fitnesses, rng)
        child = crossover(parent_a, parent_b, rng, crossover_rate)
        child = mutate(child, rng, mutation_rate, mutation_moves)
        next_population.append(child)
    population = next_population
```

Setiap generasi mencatat `max_objective`, `avg_objective`, `best_value`, `avg_value` (individu tidak valid dihitung 0), dan `feasible_count`. Kedua panel `objective.png` digambar dari data ini. Individu terbaik generasi 0 disimpan sebagai `initial_state`, sedangkan individu terbaik sepanjang run disimpan sebagai `final_state`. Jika `max_iter` = 0, loop tidak

berjalan dan GA langsung mengambil individu terbaik dari populasi awal.

BAB V
PENGUJIAN

5.1. Skenario Pengujian

Agar pengujian tetap ringkas, seluruh eksperimen hanya memakai dataset wajib `test_case` dengan parameter default `experiments.json` dan seed 1, 2, 3, sesuai jumlah minimum run yang diminta spesifikasi. Unit test di folder `tests/` dipakai untuk memastikan kebenaran implementasi.

Seluruh eksperimen dijalankan pada satu perangkat yang sama, yaitu laptop dengan prosesor Intel Core Ultra 5 125H, RAM 16 GB, sistem operasi Windows 11 Home, dan Python 3.12.0. Dengan demikian, durasi antaralgoritma dapat dibandingkan secara langsung. Nilai durasi hanya berlaku relatif terhadap perangkat ini.

Tabel 5.1 Skenario eksperimen

Eksperimen	**Konfigurasi**	**Jumlah run**
HC (4 varian)	Parameter Tabel 3.6	$4 \times 3 = 12$
SA	Parameter Tabel 3.7	3
GA eksperimen A	Populasi 50; iterasi 50 / 100 / 200	9
GA eksperimen B	Iterasi 100; populasi 20 / 50 / 100	9

Alur bilangan acak HC dan SA sama, sehingga untuk setiap seed keempat varian HC dan SA berangkat dari initial state yang sama, yaitu value 205 (seed 1), 165 (seed 2), dan 135 (seed 3). GA membangkitkan populasinya sendiri.

5.2. Dataset Uji

`test_case` berisi satu truk berukuran $6 \times 6 \times 4$ dengan kapasitas 100 dan delapan paket (dua di antaranya fragile, P2 dan P6). Total value seluruh paket 305 dan total beratnya 156. Batas atas dihitung dengan knapsack 0/1 berbasis berat (kapasitas 100) yang mengabaikan geometri, menghasilkan 205. Karena constraint geometri hanya dapat menurunkan nilai, state valid bernilai 205 pasti merupakan optimum global.

5.3. Data yang Dicatat

Tabel 5.2 Pemetaan data yang diminta spesifikasi ke output program

Diminta spesifikasi	**Lokasi pada output**
State awal dan akhir	`initial_state.json`, `final_state.json`, beserta gambar
Nilai objective akhir	`summary.json`: `final_objective`, `best_valid_value`
Plot objective terhadap iterasi	`objective.png`
Durasi	`summary.json`: `duration_s`
HC: iterasi sampai berhenti	`iterations`, `termination_reason`
HC Sideways: iterasi dan batas sideways	`params.max_consecutive_sideways`, `extra.sideways_moves`
HC Random Restart: restart dan iterasi per restart	`extra.restart_count`, `extra.iterations_per_restart`, `restart_summary.csv`
SA: plot $e^{\Delta E/T}$ dan frekuensi stuck	`accept_prob.png`, `stuck_count`, `stuck_rate`
GA: objective maksimum dan rata-rata	`objective.png`, `history.csv`

5.4. Hasil pada Test Case

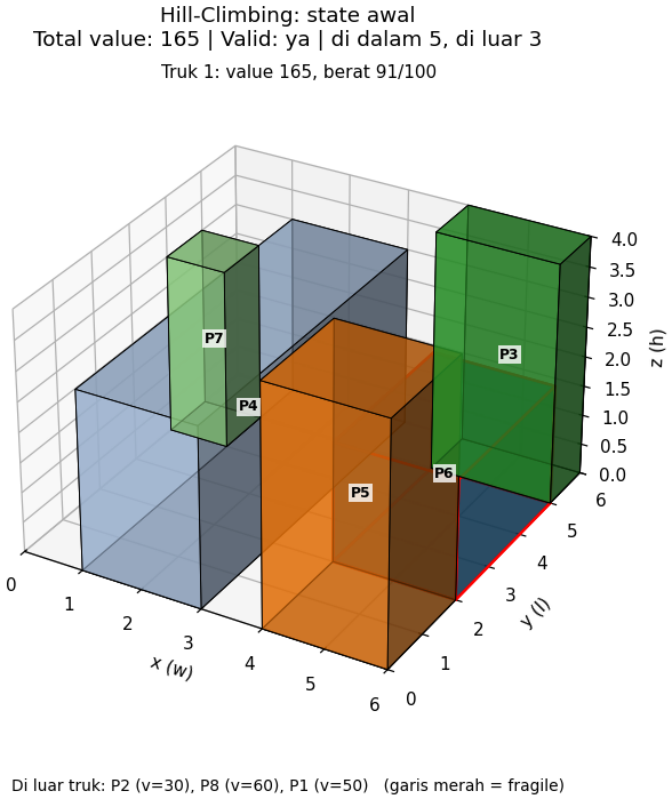
5.4.1. Hill-Climbing

Tabel 5.3 Hasil Hill-Climbing pada test_case

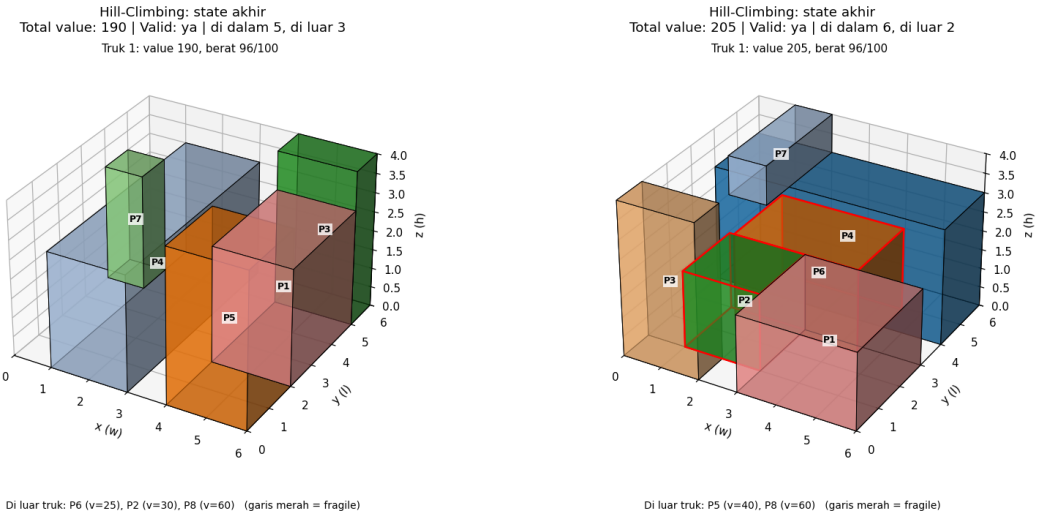
Varian	Value akhir (seed 1/2/3)	Iterasi (seed 1/2/3)	Durasi rata-rata (s)	Keterangan
Steepest Ascent	205 / 190 / 150	1 / 2 / 2	0,05	`LOCAL_OPTIMUM`
Sideways Move	205 / 190 / 185	50 / 51 / 54	1,82	`MAX_SIDEWAYS`; total sideways 50 / 50 / 51
Stochastic	205 / 190 / 150	1 / 3 / 2	0,05	`LOCAL_OPTIMUM`; langkah diterima 0 / 2 / 1
Random Restart	205 / 205 / 200	12 / 16 / 15	0,68	`MAX_RESTARTS`; 10 restart; restart terbaik 0 / 6 / 5

Pada Sideways Move, `max_consecutive_sideways` = 50 membatasi jumlah langkah datar *berturut-turut*, bukan jumlah total langkah datar selama pencarian. Penghitungnya kembali ke 0 setiap kali terjadi langkah naik. Oleh karena itu total sideways pada seed 3 bisa bernilai 51: dari 54 iterasi, 3 merupakan langkah naik dan 51 merupakan langkah datar, dengan 50 langkah datar berturut-turut di bagian akhir.

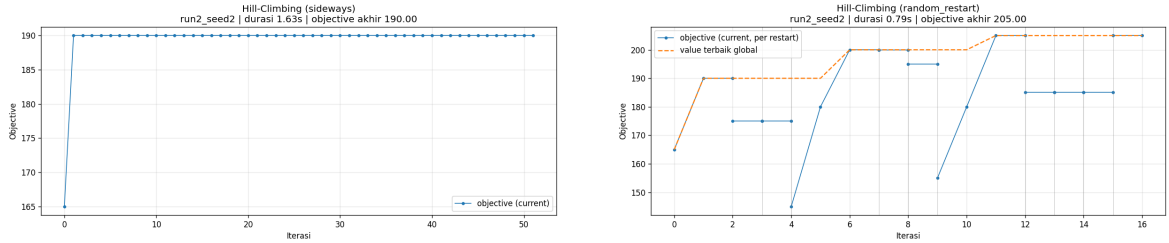
Dengan `max_restart` = 10, Random Restart menjalankan Steepest Ascent sebanyak 11 kali, yaitu satu pencarian dari initial state awal dan 10 pencarian dari initial state baru hasil restart. Iterasi per restart pada Random Restart (di luar pencarian awal) adalah [2, 1, 1, 1, 1, 1, 1, 1, 1, 1] untuk seed 1, [1, 1, 3, 1, 1, 3, 1, 1, 1, 1] untuk seed 2, dan [1, 1, 1, 1, 3, 1, 1, 1, 2, 1] untuk seed 3. Pencarian awalnya masing-masing berakhir di 205, 190, dan 150.



Gambar 5.1 State awal test_case seed 2 (value 165), dipakai bersama oleh keempat varian HC dan SA



Gambar 5.2 State akhir seed 2: Steepest Ascent (value 190, kiri) dan Random Restart (value 205, kanan)



Gambar 5.3 Objective terhadap iterasi seed 2: Sideways Move (kiri) dan Random Restart (kanan)

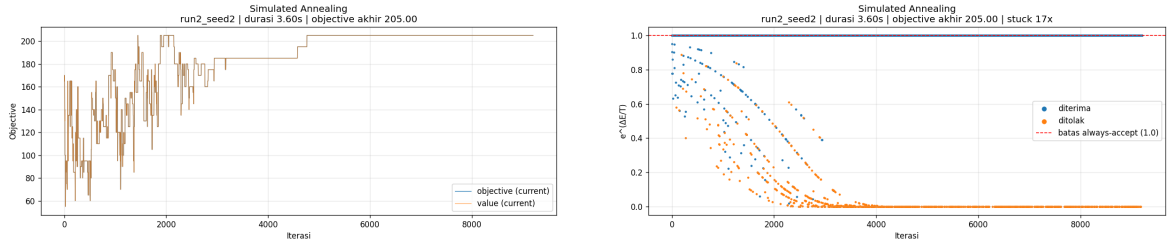
Pada seed 2, Steepest Ascent, Sideways Move, dan Stochastic sama-sama tertahan di local optimum bernilai 190. Sideways Move menghabiskan 50 langkah datar tanpa menemukan jalan naik, sedangkan Random Restart menemukan 205 pada restart ke-6.

5.4.2. Simulated Annealing

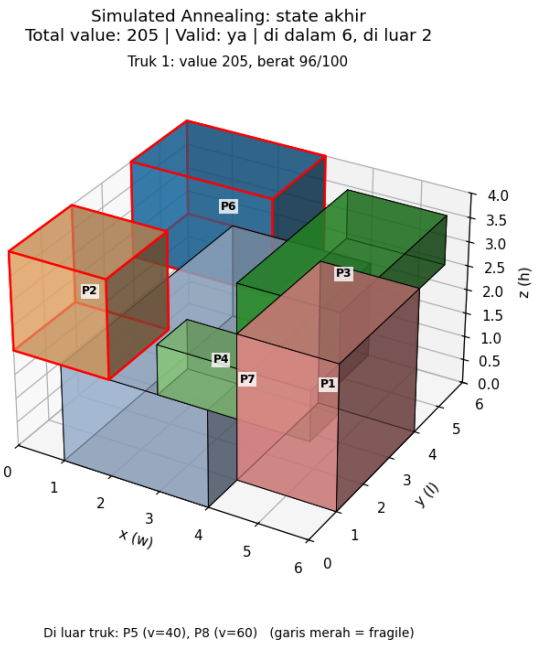
Tabel 5.4 Hasil Simulated Annealing pada test_case

Seed	Value awal	Value state akhir	best_valid	Iterasi	Stuck (per 1.000 iter)	Durasi (s)
1	205	200	205	9.206	18 (1,96)	2,76
2	165	205	205	9.206	17 (1,85)	3,60
3	135	205	205	9.206	16 (1,74)	3,65

Dari 9.206 iterasi, langkah yang menurunkan objective dan tetap diterima berjumlah 124 (seed 1), 119 (seed 2), dan 108 (seed 3). Sebagian besar langkah diterima lainnya adalah langkah datar ($\Delta E = 0$, peluang 1).



Gambar 5.4 SA seed 2: objective terhadap iterasi (kiri) dan $e^{\Delta E/T}$ terhadap iterasi (kanan)

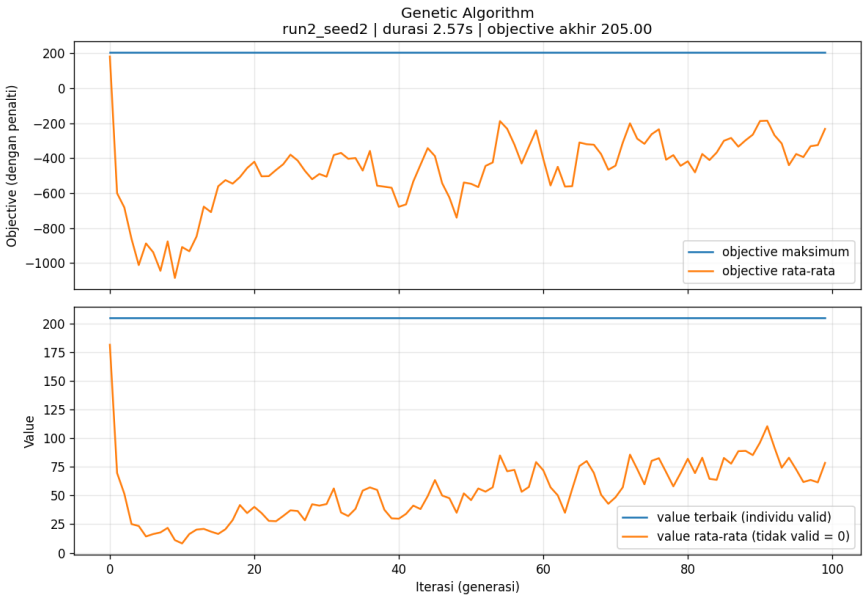


Gambar 5.5 State akhir Simulated Annealing seed 2 (value 205)

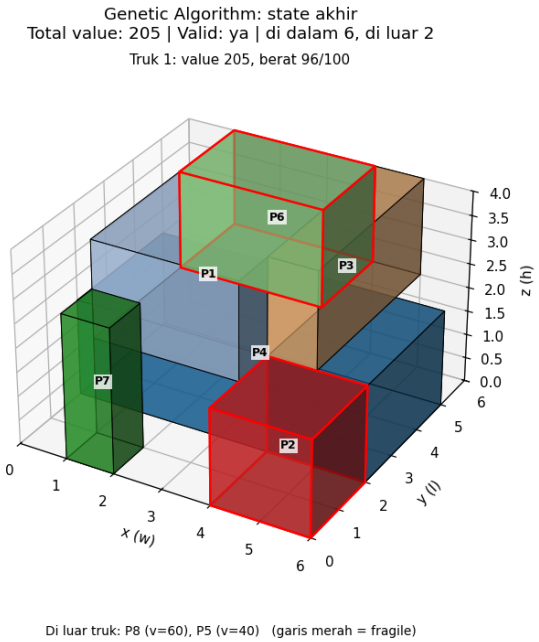
5.4.3. Genetic Algorithm

Tabel 5.5 Hasil Genetic Algorithm pada test_case

Eksp.	Populasi	Iterasi	best_valid (seed 1/2/3)	Durasi (s, seed 1/2/3)	Rata-rata (s)
A	50	50	205 / 205 / 205	1,52 / 1,75 / 1,79	1,69
A	50	100	205 / 205 / 205	2,05 / 2,46 / 2,29	2,27
A	50	200	205 / 205 / 205	3,38 / 4,10 / 3,91	3,80
B	20	100	205 / 205 / 205	1,10 / 1,09 / 1,04	1,08
B	50	100	205 / 205 / 205	2,06 / 2,57 / 2,25	2,29
B	100	100	205 / 205 / 205	4,29 / 4,15 / 4,74	4,39



Gambar 5.6 Objective maksimum dan rata-rata populasi GA (populasi 50, iterasi 100, seed 2)



Gambar 5.7 Individu terbaik GA (populasi 50, iterasi 100, seed 2; value 205)

5.5. Analisis Hasil

5.5.1. Kedekatan dengan Optimum Global

- GA mencapai optimum 205 di seluruh 18 run, dan SA di ketiga run (dilihat dari `best_valid`).
- HC Random Restart mencapai 205 pada 2 dari 3 run; seed 3 berhenti di 200.

- Steepest Ascent, Sideways Move, dan Stochastic hanya mencapai optimum pada seed 1, yang memang sudah dimulai dari state optimum.

Penerimaan langkah turun memungkinkan SA keluar dari local optimum 190 dan 150 yang menahan HC. Pada seed 3, SA naik dari 135 ke 205, sedangkan Steepest Ascent dari state yang sama berhenti di 150.

5.5.2. Perbandingan Hasil Antaralgoritma

Urutan kualitas hasil pada test_case adalah GA $\approx$ SA $>$ HC Random Restart $>$ HC Sideways $>$ HC Steepest $=$ HC Stochastic. Sideways Move sedikit lebih baik pada seed 3 (185 dibanding 150) karena langkah datar membawanya ke daerah dengan jalan naik. Stochastic memberi hasil yang sama dengan Steepest Ascent di ketiga seed: keduanya mengevaluasi seluruh neighborhood dan berhenti di local optimum yang sama, hanya jalur kenaikannya yang dapat berbeda.

5.5.3. Perbandingan Durasi

HC Steepest Ascent dan Stochastic paling cepat (sekitar 0,05 detik) karena hanya butuh 1–3 iterasi. Sideways Move paling lambat di antara varian HC (sekitar 2 detik) karena setiap langkah datar tetap mengenumerasi seluruh neighbor. SA selalu menjalankan 9.206 iterasi dengan durasi 3–4 detik. Durasi GA tumbuh hampir linear terhadap populasi dan iterasi.

5.5.4. Konsistensi Hasil

GA dan SA paling konsisten (`best_valid` 205 di semua seed). HC Steepest dan Stochastic paling bergantung pada initial state (205 / 190 / 150). Random Restart mengurangi ketergantungan tersebut dengan mengambil hasil terbaik dari 11 titik awal.

5.5.5. Pengaruh Populasi dan Iterasi pada GA

Variasi populasi dan iterasi tidak mengubah kualitas hasil pada test_case karena individu terbaik generasi 0 sudah bernilai 205: dengan hanya delapan paket, populasi acak sudah memuat optimum. Rata-rata objective populasi turun tajam setelah generasi 0 (dari 181,6 menjadi $-600{,}9$ pada generasi 1) karena crossover uniform sering menghasilkan anak tidak valid yang terkena penalti λ; jumlah individu valid turun dari 50 menjadi 20. Untuk dataset sekecil ini, populasi 20 dengan 100 iterasi sudah cukup.

5.5.6. Perilaku Simulated Annealing

Dengan penerimaan probabilistik, plot $e^{\Delta E/T}$ menunjukkan nilai yang tinggi di awal (langkah buruk sering diterima) dan mendekati nol setelah suhu turun, kecuali $\Delta E = 0$ yang selalu bernilai 1. Pada seed 1, SA sempat meninggalkan state optimum 205 dan berakhir di 200,

sehingga penyimpanan `best_valid` tetap penting. Frekuensi stuck sekitar 1,7–2 kejadian per 1.000 iterasi; angka ini juga menghitung jendela setelah optimum ditemukan, sehingga lebih tepat dibaca sebagai banyaknya jendela 500 iterasi tanpa perbaikan.

BAB VI
KESIMPULAN & SARAN

6.1. Kesimpulan

1. Persoalan pengisian peti kemas Mejiro Express berhasil diselesaikan dengan empat varian Hill-Climbing, Simulated Annealing, dan Genetic Algorithm yang berbagi representasi state, pengecekan constraint, dan operator neighbor yang sama.

2. Pada test_case, optimum global 205 dicapai oleh seluruh 18 run GA, ketiga run SA, 2 dari 3 run HC Random Restart, dan 1 dari 3 run untuk varian HC lainnya.

3. Algoritma yang mampu keluar dari local optimum (SA) atau mencoba banyak titik awal (GA dan Random Restart) secara konsisten lebih baik daripada HC biasa.

4. Variasi populasi dan iterasi GA pada test_case hanya memengaruhi durasi, bukan kualitas hasil.

5. Bonus OF prioritas ETA, keempat varian HC, dan multi-truk berhasil diimplementasikan tanpa perubahan pada kode algoritma inti.

6.2. Keterbatasan

1. Pengujian hanya dilakukan pada test_case yang berukuran kecil, sehingga perbedaan antaralgoritma, terutama pengaruh parameter GA, belum terlihat penuh.

2. Seluruh varian HC mengenumerasi seluruh neighbor, sehingga biaya per iterasinya tumbuh cepat pada truk besar dengan banyak paket.

3. Definisi melayang mengikuti spesifikasi secara harfiah (cukup satu titik alas ditopang), sehingga paket boleh menggantung sebagian.

6.3. Saran

1. Menguji ulang pada dataset yang lebih besar agar pengaruh populasi dan iterasi GA terlihat.

2. Membatasi neighborhood HC, misalnya hanya mencoba posisi yang menempel pada dinding atau paket lain.

3. Mengubah definisi stuck SA agar hanya dihitung sebelum solusi terbaik ditemukan.

DAFTAR PUSTAKA

[1] M. L. Khodra, “Beyond Classical Search: Classical vs Local Search,” YouTube, 2021. [Video].

[2] M. L. Khodra, “Beyond Classical Search: State (Value, Neighbor),” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM

[3] M. L. Khodra, “Beyond Classical Search: Hill-climbing Search,” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8

[4] M. L. Khodra, “Beyond Classical Search: Simulated Annealing,” YouTube, 2021. [Video].

[5] M. L. Khodra, “Beyond Classical Search: Genetic Algorithm,” YouTube, 2021. [Video].

[6] S. Russell dan P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

Lampiran A. Pembagian Tugas

NIM	Nama	Tugas
13524028	Muhammad Nur Majiid	Model data dan Genetic Algorithm
13524064	Stefani Angeline Oroh	Algoritma Hill-Climbing dan laporan
13524079	Angelina Andra Alanna	Fungsi objektif, Simulated Annealing, visualisasi, dan laporan

Lampiran B. Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	:	Muhammad Nur Majiid / 13524028 Stefani Angeline Oroh / 13524064 Angelina Andra Alanna / 13524079
Fakultas/Sekolah	:	Sekolah Teknik Elektro dan Informatika (STEI)
Kode Mata Kuliah	:	IF3170
Nama Mata Kuliah	:	Inteligensi Artifisial
Judul Tugas/Proposal	:	Tugas Besar 1: Local Search untuk Optimasi Pengisian Peti Kemas Truk

menyatakan bahwa kami **menggunakan** AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas. Alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Memeriksa ejaan dan tata bahasa teks laporan
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	–
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Membantu memahami salindia perkuliahan
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya	Google Gemini untuk membuat ilustrasi constraint dan orientasi paket (Gambar 1.1 dan 1.2) agar penjelasan tertulis lebih mudah divisualisasikan
Lainnya, Jelaskan:	–	–

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026





Muhammad Nur Majiid	Stefani Angeline Oroh	Angelina Andra Alanna
13524028	13524064	13524079

Lampiran C. Tautan Repository

`https://github.com/MAJIIDMN/IF3170_G15_WedaBest`